Peer Response

by Abdulrahman Alhashmi - Thursday, 4 September 2025, 10:45 AM

Saleh, your post offered great insights. It clearly explained KQML's theory. You focused on meaning and context in agent talk. Your point was that KQML goes beyond just sending messages. It builds meaning into conversations. This is vital for different systems working together. This helps them connect. It's a big plus for areas like online shopping. It also helps supply chains. Agents on different systems must work well. (Singh and Huhns, 2005).

Your point about computing cost and difficulty is vital. Advanced thinking needs significant system power. As you noted, slow speed can prevent real-time use. This often makes calling Python or Java methods seem better. This is true where predictable and fast action is key.(Bellifemine, Caire and Greenwood, 2007).

Ontologies are crucial for effective ACLs. A common vocabulary prevents confusion and communication errors (Wooldridge, 2009). Method invocation offers a simpler approach. This clarity avoids ambiguity but sacrifices adaptability.

The comparison reveals no single answer. ACLs grant independence and scale in open systems. Method calls work best in unified, linked systems. Future study could explore mixed systems. These would blend ACLs' deep meaning with method calls' speed.

The examination effectively highlights the main discussion. It also prompts consideration of real-world uses.

References

Bellifemine, F., Caire, G. and Greenwood, D., 2007. Developing multi-agent systems with JADE. John Wiley & Sons.

Singh, M.P. and Huhns, M.N., 2005. Service-oriented computing: semantics, processes, agents. John Wiley & Sons.

Wooldridge, M., 2009. An introduction to multiagent systems. 2nd ed. Wiley.

Peer Response

by Abdulrahman Alhashmi - Friday, 5 September 2025, 3:29 PM

Shaikah, your post was clear and organised. You outlined the pros and cons of agent communication languages (ACLs), like KQML. Your comparison of ACLs to method calls in Python and Java was very helpful. I agree that ACLs allow for deeper, meaning-based agent talks. This supports agent independence and bargaining. Such flexibility is vital for scattered systems. Agents need to work alone but still well together (Singh, 1998).

The complexity you noted matches what experts have found. ACLs can slow systems and cause mix ups. This happens if meanings are not clearly shared (Wooldridge, 2009). This issue often stops ACLs from being used widely, even if they seem good in theory. Method calls, however, offer ease and trust. This is true when systems are linked closely and speed is key. Method calls suit apps like business systems or built in tools. There, speed matters more than deep thought (Jennings and Bussmann, 2003).

A key aspect is how these two approaches work together. ACLs enable smart and adaptable systems. Method calls offer dependability and speed. The ideal option often hinges on system demands. Does the system require independence and easy changes? Or is a simple and fast setup preferable? Your piece highlights this trade-off well. It raises an interesting question for further research: can we streamline ACLs, or enhance method call flexibility.

References

Jennings, N.R. and Bussmann, S. (2003) ‘Agent-based control systems: Why are they suited to engineering complex systems?’, IEEE Control Systems Magazine, 23(3), pp. 61–73.

Singh, M.P. (1998) ‘Agent communication languages: Rethinking the principles’, IEEE Computer, 31(12), pp. 40–47.

Wooldridge, M. (2009) An introduction to multiagent systems. 2nd edn. Chichester: Wiley.

Peer Response

by Abdulrahman Alhashmi - Saturday, 6 September 2025, 4:29 PM

Your post effectively detailed the pros and cons of Agent Communication Languages (ACLs). You rightly highlighted their main advantages: abstraction and interoperability. I agree that ACLs communicate more than just data; they convey intent. This ability suits independent agents in various environments (Singh and Huhns, 2005). Simple method calls struggle to match this depth of meaning.

However, I also worry about the requirement for shared ontologies. Making sure different agents understand things the same way remains a tough problem in distributed systems. Studies show that a lack of shared meaning often hinders teamwork. This makes multi-agent systems more costly and complex (Gruber, 2009). This major drawback is a world away from how simple Python or Java method calls work.

Performance means more than just being quick. Python and Java method calls are good for finding bugs. They have big user groups and show how a system is built (van Rossum and Drake, 2009). These helpful points make method calls reliable for tough needs. Being flexible matters less then. ACLs are great for online shops. They also work for big computer models or robots. Being able to act alone and talk is more important than speed here (Jennings and Bussmann, 2003).

Basically, the main choice is clear. ACLs are good for being flexible and acting by yourself. Method calls are best for speed and simple use. Builders must pick carefully. The best choice depends on making a system that is open or shut.

References

Gruber, T.R., 2009. Collective knowledge systems: Where the social web meets the semantic web. Web Semantics: Science, Services and Agents on the World Wide Web, 6(1), pp.4–13.

Jennings, N.R. and Bussmann, S., 2003. Agent-based control systems: Why are they suited to engineering complex systems? IEEE Control Systems Magazine, 23(3), pp.61–73.

Singh, M.P. and Huhns, M.N., 2005. Service-oriented computing: Semantics, processes, agents. Chichester: Wiley.

van Rossum, G. and Drake, F.L., 2009. Python 3 reference manual. Scotts Valley: CreateSpace.